

Introduction aux Design Patterns

PARTIE DU COURS DE XAVIER CRÉGUT

DÉPARTEMENT TÉLÉCOMMUNICATIONS & RÉSEAUX ENSEEIHT

A solid orange horizontal bar at the bottom of the slide.

Historique

- Notion de « patron » d'abord apparue en architecture :
 - ▶ Christopher Alexander : « The Timeless Way of Constructing », 1979
 - ▶ Définition de « patrons » pour :
 - ★ l'architecture des bâtiments
 - ★ la conception des villes et de leur environnement
 - ▶ Christopher Alexander :
« Chaque modèle [patron] décrit un **problème qui se manifeste constamment** dans notre environnement, et donc **décrit le cœur de la solution** de ce problème, d'une façon telle que l'on peut **réutiliser cette solution** des millions de fois, sans **jamais le faire deux fois de la même manière.** »
- Idée : appliquer la notion de patron à du logiciel : « design patterns »
 - ▶ premiers patrons à partir de 1987 (partie de la thèse de Erich Gamma)
 - ▶ puis Richard Helm, John Vlissides et Ralph Johnson (« Gang of Four, GoF »)
 - ▶ 1er catalogue en 1993 : *Elements of Reusable Object-Oriented Software*
- Vocabulaire :
 - ▶ design patterns
 - ▶ patrons de conception
 - ▶ modèles de conception
 - ▶ micro-architectures

Intérêt des patrons de conception

Pourquoi définir des patrons de conception

- Construire des systèmes plus extensibles, plus robustes au changement
- Capitaliser l'*expérience collective* des informaticiens
- Réutiliser les solutions qui ont fait leur preuve
- Identifier les avantages/inconvénients/limites de ces solutions
- Savoir quand les appliquer

Complémentaire avec les API

- une API propose des solutions directement utilisables
- un patron explique comment structurer son application, une API

Patron de conception dans le cycle de développement

- intervient en conception détaillée
- reste indépendant du langage d'implantation

Qu'est ce qu'un patron de conception ?

- **Définition** : Un *patron de conception* (*design pattern*) décrit une structure commune et répétitive de composants en interaction (la **solution**) qui résout un **problème de conception** dans un **contexte particulier**.
- **Quatre éléments essentiels**
 - ▶ **nom** : un ou deux mots pour décrire le problème de conception considérée, ses solutions et leurs conséquences.
 - ▶ **problème** : situation où le problème s'applique
 - ▶ **solution** : éléments de la conception, leurs relations et collaborations.
 - ★ la solution n'est pas forcément précise : idée d'architecture.
 - ★ plusieurs variantes peuvent être possibles.
 - ▶ **conséquences** : effets résultants et compromis induits
 - ★ Les conséquences peuvent être positives ou négatives (arbitrage).
- Un **bon patron de conception** :
 - ▶ résout un problème
 - ▶ correspond à une solution éprouvée
 - ▶ favorise la réutilisabilité, l'extensibilité, etc.
 - ▶ inclut une composante subjective : utilité, esthétique, etc.

Classification des patrons de conception

Fondamental	Créateur	Structurel	Comportement
	Fabrique	Adaptateur	Interprète
Délégation	Fabrique abstraite	Adaptateur	Patron de méthode
Interface	Monteur	Pont	Chaîne de responsabilités
Classe abstraite	Prototype	Composite	Commande
Immuable	Singleton	Décorateur	Itérateur
Interface de marquage		Façade	Médiateur
		Poids mouche	Memento
		Procuration	Observateur
			État
			Stratégie
			Visiteur

- **fondamental** : application directe des concepts objets ;-)
- **créateur** : processus de création d'objets
- **structurel** : architecture statique du système
- **comportemental** : interactions entre objets et répartition des responsabilités

Patrons fondamentaux

Délégation

- **Contexte :** Parfois, utiliser l'héritage conduit à une mauvaise conception. La délégation est alors un mécanisme plus général (même si plus lourd).
 - **Cas où ne pas utiliser l'héritage (mais la délégation) :**
 - ▶ *Une classe métier veut réutiliser une classe « utilitaire ».*
 - ★ Évolutions de l'« utilitaire » compatibles avec la classe « métier » ?
 - ▶ *Une classe veut cacher des éléments de la superclasse (Anti-patron !).*
 - ★ C'est impossible à faire (si héritage implique sous-typage, ex. Java) !
Exemple : Pile ne doit pas hériter de ArrayList.
 - ▶ *Les sous-classes modélisent des rôles différents de la superclasse.*
Exemple : Modéliser un joueur de foot.
Spécialiser Joueur en Gardien, Stoppeur, Attaquant... ou définir une délégation sur Poste qui est spécialisée en Gardien, Stoppeur, Attaquant...
Un joueur peut changer de poste, voire occuper plusieurs postes.
- Remarque :** L'héritage correspond à « est une sorte de » mais n'est pas adapté pour représenter « est un rôle joué par » car, sinon, le rôle ne peut pas changer pendant la durée de vie de l'objet.

Patrons fondamentaux

Interface

- **Intention :** On souhaite qu'un client reste indépendant de la classe qui fournit effectivement le service.
- C'est justement l'objectif des interfaces ! (patron fondamental)
- **Exemple :** Un client veut accéder à un fournisseur JDBC pour accéder à une base de données. Le fournisseur effectif dépendra de la BD considérée. Le code client n'a pas à le connaître, seulement les interfaces qui définissent les services JDBC.
- **Intérêt :** Le fournisseur de service peut changer sans impact sur le code client.

Patrons fondamentaux

Classe abstraite

- **Intention :**

- ▶ Garantir que la logique commune à plusieurs classes est implantée de manière cohérente pour chaque classe
- ▶ Éviter le code redondant (et les efforts de maintenance associés)
- ▶ Faciliter l'écriture de nouvelles classes implantant la même logique

- **Solution :**

- ▶ Utiliser une classe abstraite pour factoriser le code commun implantant la logique
- ▶ Les méthodes peuvent éventuellement être définies comme **final**

- **Exemple :** La classe abstraite `java.util.AbstractCollection` définit toutes les opérations de `Collection` sauf `size` et `iterator`.

- ▶ Collection concrète non modifiable : définir seulement `size` et `iterator`
- ▶ Collection concrète modifiable : définir aussi `add` (et `remove` sur l'iterator).

Exemple 2 : La classe `TestCase` de JUnit (cf *Patron de méthode*, T. 109)

Patrons fondamentaux

Immuable

- **Intention :**

- ▶ Augmenter la robustesse des objets partageant les mêmes objets
- ▶ Diminuer le coût de la gestion du parallélisme.

- **Intérêts :**

- ▶ facilite la gestion des objets partagés (personne ne peut les modifier !)
- ▶ permet de réaliser une composition sans avoir à faire de copie des objets
- ▶ pas de besoin de synchronisation (lecture seule)
- ▶ MAIS toute « modification » nécessite la création d'un nouvel objet

- **Exemple :**

- ▶ String (immuable) par opposition à StringBuffer (modifiable).

Patrons fondamentaux

Interface de marquage

- **Intention** : Définir une propriété sémantique booléenne sur une classe
 - **Solution** : Hériter d'une interface de marquage (interface vide).
 - **Exemples** :
 - ▶ Cloneable indique que les objets d'une classe peuvent être clonés.
 - ▶ Serializable indique qu'un objet peut être sérialisé¹
 - **Forces** :
 - ▶ Tester une propriété d'un objet/classe sans connaître la classe réelle
- ```
1 Object copie = null;
2 if (o instanceof Cloneable) {
3 copie = o.clone();
4 } else {
5 throw new CloneNotSupportedException();
6 }
```
- ▶ MAIS La propriété ne peut jamais être supprimée/désactivée (obligatoirement héritée par les sous-classes)

# Design Pattern : « LE COMPOSITE »

# Composite

- **Intention**

- ▶ Compose des objets en des structures arborescentes pour représenter des hiérarchies composant/composé.
- ▶ Permet au client de traiter uniformément un objet individuel ou les combinaisons de ceux-ci

- **Alias** : —

- **Indications d'utilisation**

- ▶ représentation de structures récursives d'éléments hétérogènes
- ▶ traitement uniforme d'un objet individuel ou d'une combinaison d'objets

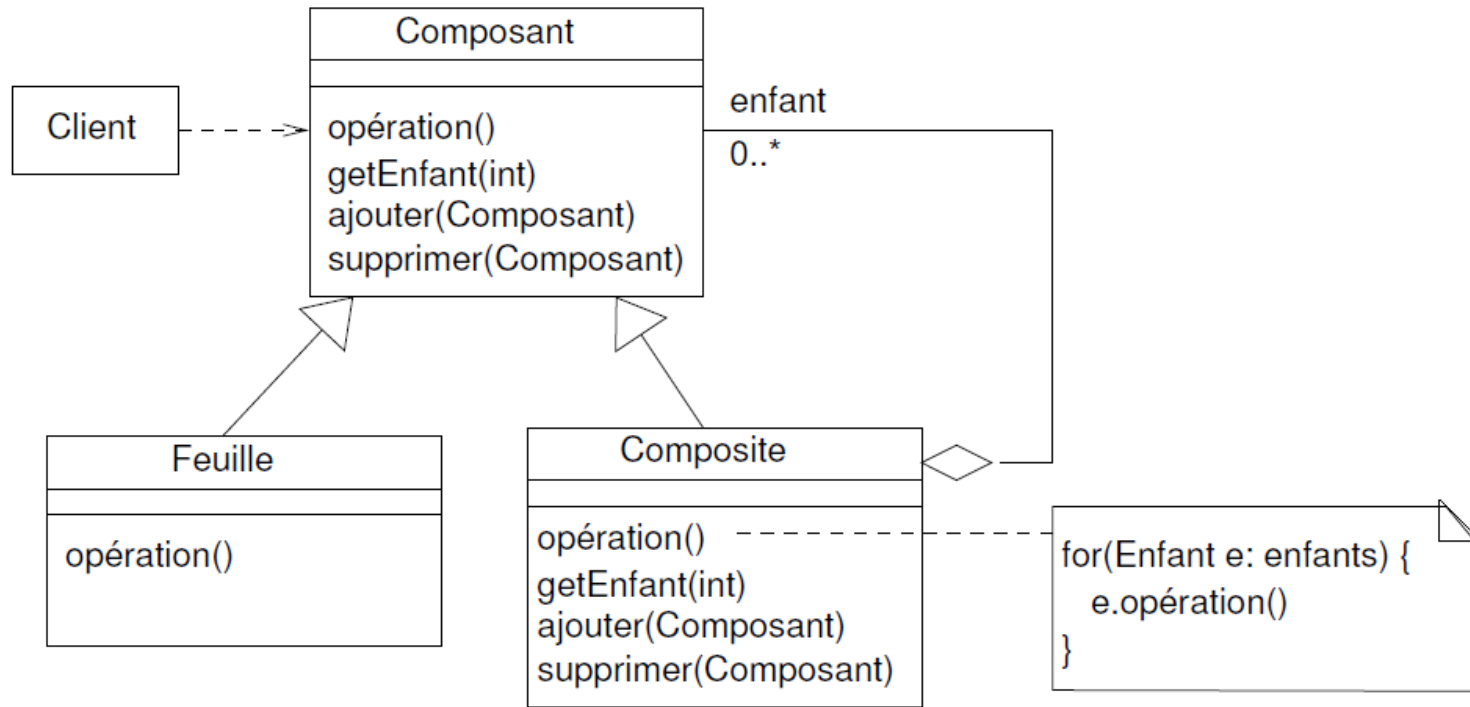
- **Exemples** : Groupe dans éditeur de schémas mathématiques.

- **Conséquences** :

- + facilite l'ajout de nouveaux types de composants
- difficile d'imposer des contraintes sur les compositions possibles

# Composite

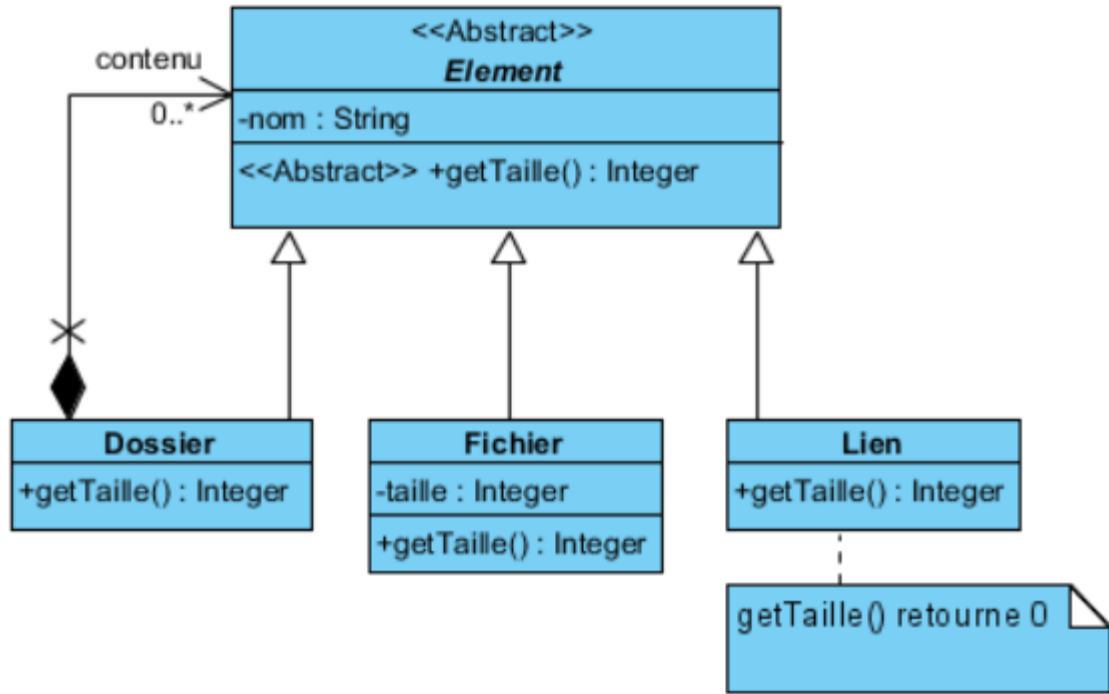
## Diagramme de classe



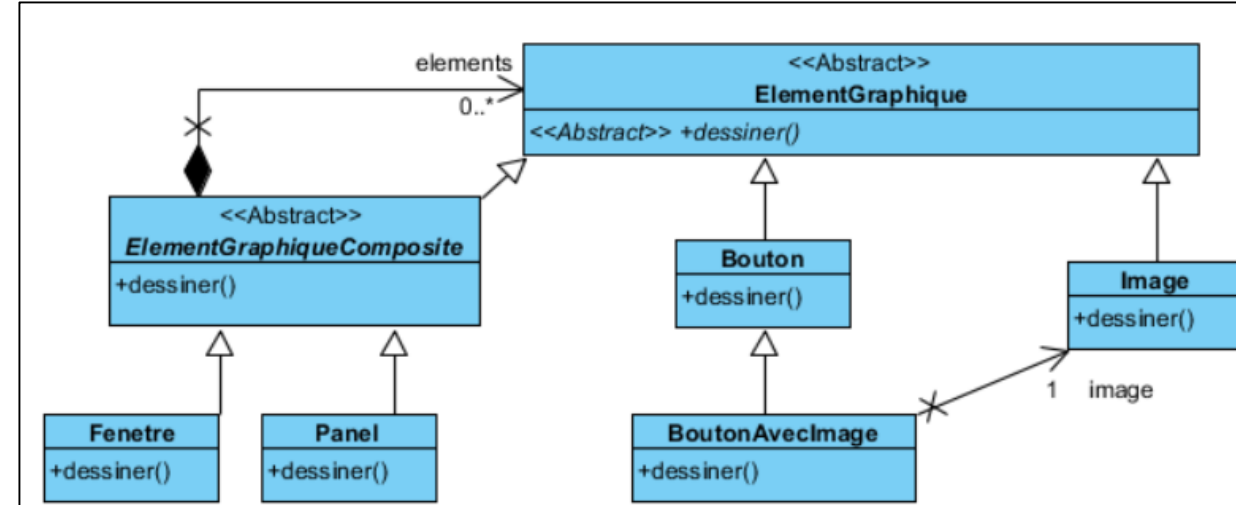
**Attention :** les opérations manipulant les composés peuvent ou non être remontées au niveau du composant : compromis entre sécurité et transparence (pour le client)

**Exemple :** Groupe d'objets géométriques.

# Composite



Exemple : Système de fichier



Exemple : Bibliothèque graphique

# Design Pattern : « LE SINGLETON »



# Singleton

- **Intention**

Garantir qu'une classe n'a qu'*une seule instance* et fournir un *point d'accès* global à cette instance.

- **Alias** : —

- **Indications d'utilisation**

- ▶ Il doit y avoir exactement une instance d'une classe
- ▶ Cette instance doit être accessible globalement
- ▶ Le type du singleton peut-être sous-classé

⇒ la classe elle-même est responsable de gérer l'unicité de l'objet et son accès

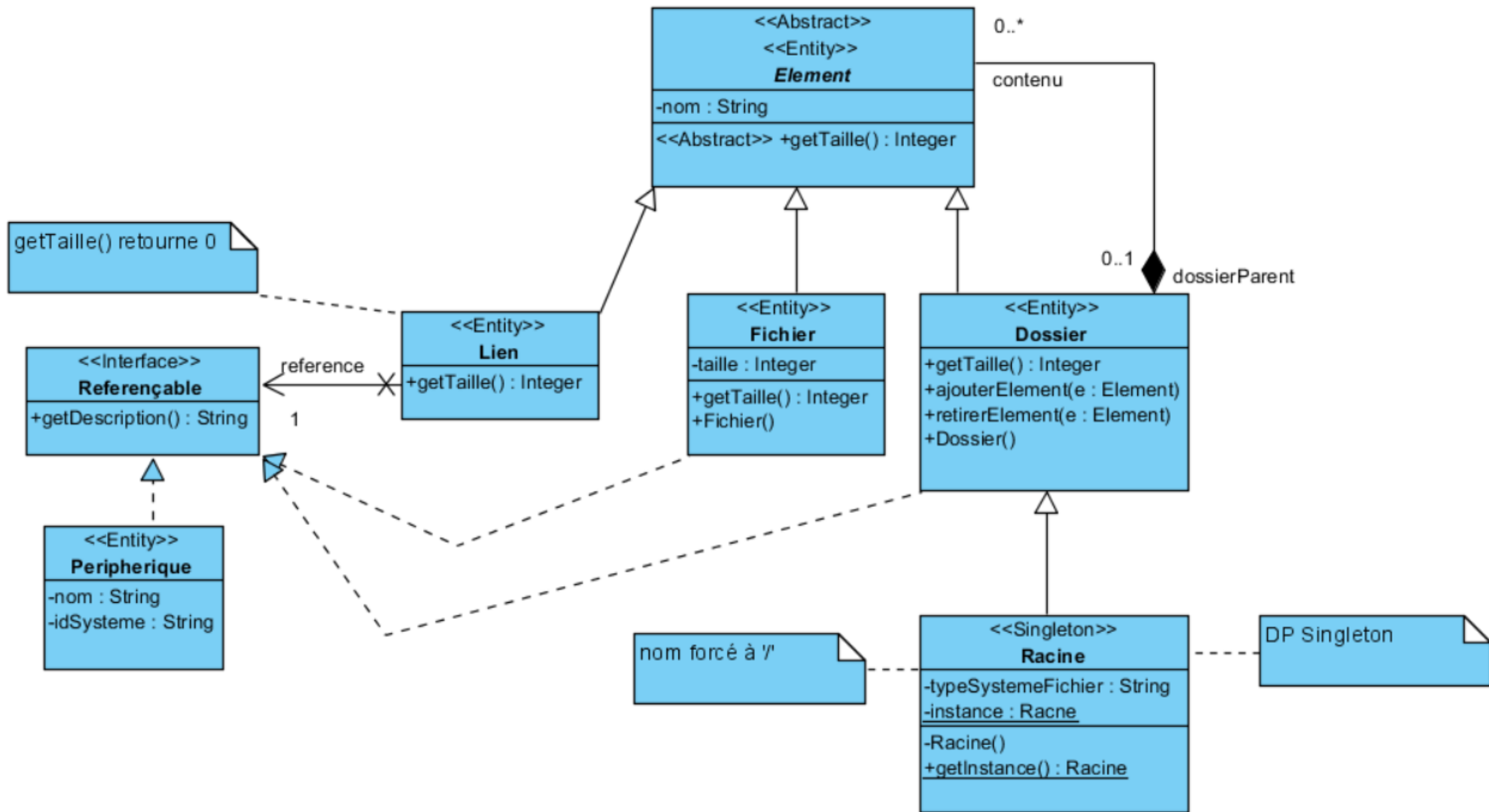
- **Exemples** : gestionnaire de fenêtres, file d'impression, etc.

# Singleton

## Diagramme de classe

| Singleton                                                |
|----------------------------------------------------------|
| <u>- instanceUnique: Singleton</u><br>- donnéesSingleton |
| + <u>getInstance(): Singleton</u><br>+ operation(...)    |
| - Singleton()                                            |

```
1 /** Manière simple et traditionnelle de faire
2 * un Singleton en Java.
3 * Attention : non sûre si utilisation de threads
4 */
5 public class Singleton {
6 private final static Singleton INSTANCE =
7 new Singleton();
8
9 // Interdire toute création d'instance
10 // depuis une classe extérieure
11 private Singleton() {}
12
13 // Accès à l'instance unique
14 public static Singleton getInstance() {
15 return INSTANCE;
16 }
17
18 }
```



# Design Pattern : « L'OBSERVATEUR »

# Observateur (Observer)

- **Intention**

Définit une interdépendance de type un à plusieurs de sorte que quand un objet change d'état, tous ceux qui en dépendent en soient avertis et automatiquement mis à jour

- **Alias :** Dépendants, Diffusion-Souscription (Publish-Subscribe)

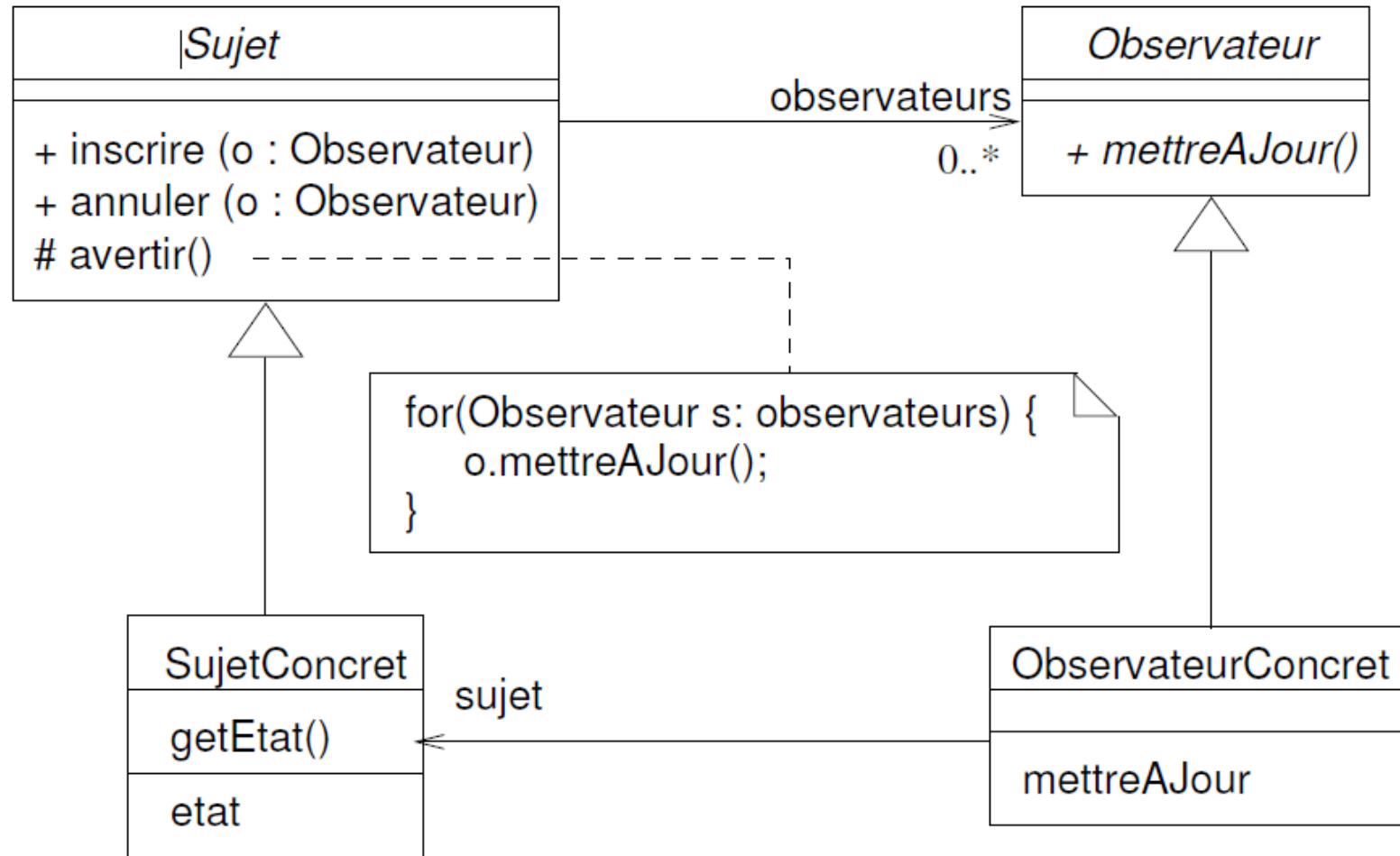
- **Indications d'utilisation**

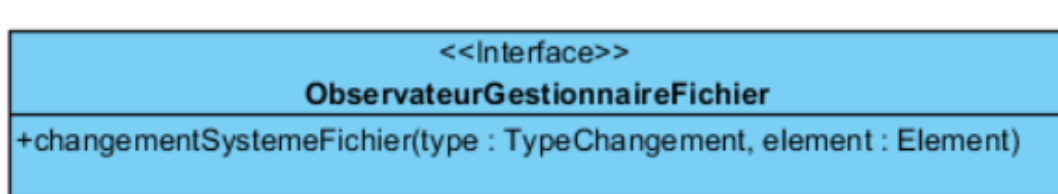
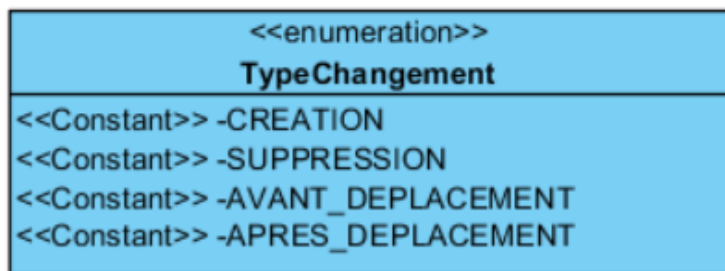
- ▶ Pour faciliter la réutilisation de deux représentations inter-dépendantes d'un même concept
- ▶ Quand la modification d'un objet nécessite de modifier un nombre indéterminé d'autres objets
- ▶ Quand un objet doit notifier d'autres objets sans faire d'hypothèses sur la nature de ces objets (faible couplage)

- **Exemples :** Plusieurs vues sur un même modèle

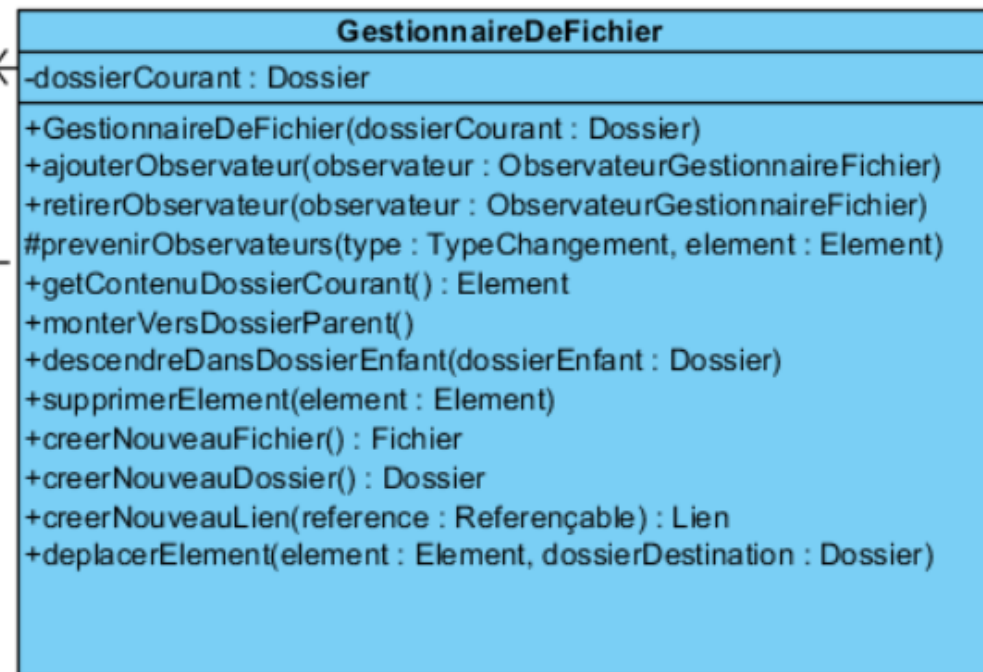
# Observateur (Observer)

Diagramme de classe

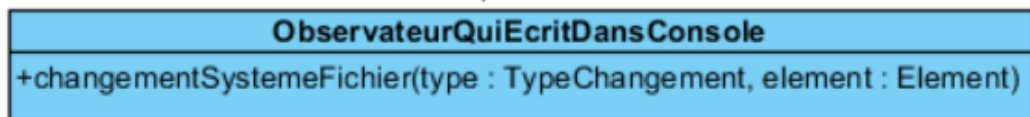




observateurs  
0..\*



Pour Chaque Observateur o  
o.changementSystemeFichier(t, e)  
Fin Pour



# Design Pattern : « LE VISITEUR »



# Visiteur (Visitor)

- **Intention**

Modélise par une classe une opération applicable aux éléments d'une structure d'objets et permet de définir de nouvelles opérations sans modifier les classes de la structure

- **Alias :** —

- **Indications d'utilisation**

- ▶ Une structure d'objets contient beaucoup de classes différentes d'interfaces distinctes et l'on veut réaliser des traitements qui dépendent de leurs classes concrètes.
- ▶ Il s'agit de réaliser plusieurs traitements distincts sans relation entre eux, sur les objets de la structure, sans polluer leurs classes
- ▶ Les classes qui définissent la structure d'objet changent rarement mais on doit souvent définir de nouvelles opérations sur cette structure.

- **Exemples :** Afficher en infixe, préfixe et postfixe une expression entière, calculer sa valeur, etc.

# Visiteur (Visitor)

Diagramme de classe

